



400-650-7088

***Tong*LINK/Q-CN** V10.0

JMS 客户端开发手册

版本: 10041A01

2025 年 9 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（10041A01）适用于TongLINK/Q-CN V10.0.4.1。

文档版本	适用产品版本	更新内容	更新日期
10041A01	V10.0.4.1	更新如下内容： 更新版权声明。	2025/9/23
10040A01	V10.0.4.0	无	2025/5/23
10030A01	V10.0.3.0	TongLINK/Q-CN V10.0.3.0第一次正式发布。	2024/11/30

前言

本文档是TongLINK/Q-CN V10.0产品的用户使用手册之一，主要介绍如何将适配ActiveMQ的JMS客户端迁移为适配TongLINK/Q-CN的JMS客户端。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

• 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

• 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

• 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握Linux系统基本操作；
2. 掌握K8s的搭建和使用；
3. 熟悉Java开发语言。

• 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

• 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户使用手册集

TongLINK/Q-CN V10.0为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongLINK/Q-CN_V10.0用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongLINK/Q-CN_V10.0快速使用手册	提供产品介绍，在K8s和虚拟机上安装部署产品的基本步骤及验证方式。

编号	手册集文档	说明
3	002_TongLINK/Q-CN_V10.0配置参数手册	提供配置文件及参数的详细说明。
4	003_TongLINK/Q-CN_V10.0监控管理手册	提供在K8s和虚拟机上安装部署监控管理组件的基本步骤及验证方式。
5	004_TongLINK/Q-CN_V10.0管理控制台使用手册	详细介绍管理控制台的集群管理、集群监控、告警管理和用户管理相关功能的使用说明。
6	005_TongLINK/Q-CN_V10.0命令行使用手册	详细提供产品中主要命令行的作用、相关参数及使用示例。
7	006_TongLINK/Q-CN_V10.0Java客户端开发手册	介绍使用Java客户端生产消费消息的相关概念、配置、参数及示例。
8	007_TongLINK/Q-CN_V10.0Kafka协议部署手册	提供在K8s和虚拟机上安装部署Kafka协议处理程序的基本步骤及验证方式。
9	008_TongLINK/Q-CN_V10.0AMQP协议部署手册	提供在K8s和虚拟机上安装部署AMQP协议处理程序的基本步骤及验证方式。
10	009_TongLINK/Q-CN_V10.0JMS客户端开发手册	介绍如何将适配ActiveMQ的JMS客户端迁移为适配TongLINK/Q-CN的JMS客户端。
11	010_TongLINK/Q-CN_V10.0MQTT协议部署手册	提供在K8s和虚拟机上安装部署MQTT协议处理程序的基本步骤及验证方式。
12	011_TongLINK/Q-CN_V10.0服务端进阶配置指南	提供在K8s和虚拟机上开启TongLINK/Q-CN的国密通信、消息轨迹、消息过滤等高级功能特性的方法。
13	012_TongLINK/Q-CN_V10.0灾备及故障转移手册	介绍使用灾备及故障转移的相关概念、配置、参数及示例。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名

- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 简介	8
2. 使用注意事项	9
2.1. JMS版本支持	9
2.2. 支持ActiveMQ的功能及限制	9
2.3. 使用建议	9
3. 业务验证	11
3.1. JMS1.1	11
3.1.1. 点对点模式	11
3.1.1.1. 引入maven依赖	11
3.1.1.2. 客户端factory	11
3.1.1.3. 发送消息	11
3.1.1.4. 消息发送验证	12
3.1.1.5. 订阅消息	12
3.1.1.6. 订阅消息验证	13
3.1.1.7. 手动消息确认	14
3.1.2. 发布订阅模式	15
3.1.2.1. 引入maven依赖	15
3.1.2.2. 客户端factory	15
3.1.2.3. 发送消息	16
3.1.2.4. 消息发送验证	16
3.1.2.5. 订阅消息	17
3.1.2.6. 订阅消息验证	18
3.1.3. 事务	18
3.1.3.1. 生产者开启事务	19
3.1.3.2. 消费者开启事务	20
3.2. JMS2.0	21
3.2.1. 点对点	21
3.2.1.1. 引入maven依赖	21
3.2.1.2. 客户端factory	21
3.2.1.3. 发送消息	21
3.2.1.4. 消息发送验证	22
3.2.1.5. 订阅消息	22
3.2.1.6. 订阅消息验证	23

3.2.1.7. 手动消息确认	23
3.2.2. 发布订阅模式	25
3.2.2.1. 引入maven依赖	25
3.2.2.2. 客户端factory	25
3.2.2.3. 发送消息	26
3.2.2.4. 消息发送验证	26
3.2.2.5. 订阅消息	27
3.2.2.6. 订阅消息验证	28
附录 A: 术语说明	29

1. 简介

ActiveMQ是一款严格遵循JMS协议的中间件，客户端开发时基于JMS协议的API，确保与ActiveMQ进行数据通信。为了将原有的ActiveMQ客户端快速改造为支持TongLINK/Q-CN，现提供一种创建遵循JMS协议的TongLINK/Q-CN客户端的方法。由于两者均遵循JMS协议，ActiveMQ客户端只需进行少量改动，大部分原有代码可复用，从而实现在尽量减少改动的情况下，快速支持TongLINK/Q-CN，实现从ActiveMQ到TongLINK/Q-CN的平滑迁移。

2. 使用注意事项

2.1. JMS版本支持

- 支持JMS1.1和JMS2.0版本的协议；
- 支持javax.jms两个版本的API；
- 不支持jakarta.jms版本的API。

2.2. 支持ActiveMQ的功能及限制

- **支持的功能：**
 1. 点对点模式下的消息发送和接收；
 2. 发布订阅模式下的消息发送和接收；
 3. 点对点模式下的消息手动确认；
 4. 单客户端事务。
- **不支持的功能：**
 1. 死信队列；
 2. AUTO, OpenWire, AMQP, Stomp, MQTT（如需支持AMQP，请参考《008_TongLINK/Q-CN_V10.0AMQP协议部署手册》）；
 3. 鉴权；
 4. 延时消息；
 5. TTL；
 6. 消息优先级；
 7. 授权；
 8. 分布式事务；
 9. 持久化订阅。

2.3. 使用建议

1. JMS协议有1.1和2.0两个版本，2.0版本完全兼容1.1版本，事务API仅提供1.1版本API。
2. jakarta.jms是当前java官方提供的最新JMS协议API包，但由于API类名的更改，所以com.datastax.oss并不支持，请使用javax.jms包下的API。使用spring相关JMS包需要注意集成的是jakarta.jms还是javax.jms。
3. javax.jms支持的客户端事务和服务端事务设置无关。仅支持单个客户端单线程事务，不支持分布式事务。
4. 开启客户端事务时，Producer客户端需要调用**session.commit()**进行session的提交。
5. 在未开启事务时，可以不设置factory的**jms.emulateTransactions**参数，因为其默认值为**false**；但若需要开启事务，则必须明确设置该参数，否则仅设置session或context的事务参数将会导致错误。

6. 消息确认仅在点对点模式且未开启事务时生效。由于事务的优先级高于消息确认，一旦消费者开开启了事务，即使进行了手动消息确认提交，也不会从队列中消费消息。

3. 业务验证

3.1. JMS1.1

3.1.1. 点对点模式

3.1.1.1. 引入maven依赖

```
<dependency>

    <groupId>com.datastax.oss</groupId>

    <artifactId>pulsar-jms-all</artifactId>

    <version></version>

</dependency>
```

3.1.1.2. 客户端factory

```
Map<String, Object> properties = new HashMap<>();

properties.put("brokerServiceUrl", "pulsar://服务器IP:6650");

properties.put("webServiceUrl", http://服务器IP:8080);

PulsarConnectionFactory factory = new PulsarConnectionFactory(properties);
```

说明:

- brokerServiceUrl: 用于设置TongLINK/Q-CN Broker的端口和地址，如果是集群部署，可用逗号隔开；
- webServiceUrl: 用于设置TongLINK/Q-CN服务对外提供的http连接地址，如果是集群部署，可用逗号隔开。

3.1.1.3. 发送消息

```
Connection connection = factory.createConnection();

try {

    Session session = connection.createSession();
```

```

Destination destination = session.createQueue(topic_name);

MessageProducer messageProducer = session.createProducer(destination);

TextMessage textMessage = session.createTextMessage("");

messageProducer.send(textMessage);

} finally {

    connection.close();

}

```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.1.1.4. 消息发送验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否发送成功，**msgInCounter**为当前发送的消息数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t

```

{
  "msgRateIn" : 0.0,
  "msgThroughputIn" : 0.0,
  "msgRateOut" : 0.0,
  "msgThroughputOut" : 0.0,
  "bytesInCounter" : 744,
  "msgInCounter" : 4,
  "bytesOutCounter" : 2232,
  "msgOutCounter" : 12,
  "averageMsgSize" : 0.0,
  "msgChunkPublished" : false,
  "storageSize" : 744,
  "backlogSize" : 186,
  "publishRateLimitedTimes" : 0,
  "earliestMsgPublishTimeInBacklogs" : 0,
  "offloadedStorageSize" : 0,
  "lastOffloadLedgerId" : 0,
  "lastOffloadSuccessTimeStamp" : 0,
  "lastOffloadFailureTimeStamp" : 0,
  "ongoingTxnCount" : 0,
  "abortedTxnCount" : 0,
  "committedTxnCount" : 0,
  "totalTxnCount" : 0
}

```

3.1.1.5. 订阅消息

生成者在发送消息的时候，会自动开启连接，但消费者需要手动开启连接。

```
try {

    Connection connection = factory.createConnection();

    connection.start();

    Session session = connection.createSession();

    Destination destination = session.createQueue(topic_name);

    MessageConsumer messageConsumer = session.createConsumer(destination);

    messageConsumer.setMessageListener(new MessageListener() {

        @Override

        public void onMessage(Message message) {

            try {

                //消息处理

            } catch (JMSEException e) {

                throw new RuntimeException(e);

            }

        }

    });

} catch (JMSEException e) {

    // TODO Auto-generated catch block

}
```

说明：

topic_name实例：persistent://test-t/test-n/test-t

3.1.1.6. 订阅消息验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否成功消费，**msgOutCounter**为当前消息消费的数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t

```
{
  "msgRateIn" : 0.0,
  "msgThroughputIn" : 0.0,
  "msgRateOut" : 0.0,
  "msgThroughputOut" : 0.0,
  "bytesInCounter" : 744,
  "msgInCounter" : 4,
  "bytesOutCounter" : 2232,
  "msgOutCounter" : 12,
  "averageMsgSize" : 0.0,
  "msgChunkPublished" : false,
  "storageSize" : 744,
  "backlogSize" : 186,
  "publishRateLimitedTimes" : 0,
  "earliestMsgPublishTimeInBacklogs" : 0,
  "offloadedStorageSize" : 0,
  "lastOffloadLedgerId" : 0,
  "lastOffloadSuccessTimeStamp" : 0,
  "lastOffloadFailureTimeStamp" : 0,
  "ongoingTxnCount" : 0,
  "abortedTxnCount" : 0,
  "committedTxnCount" : 0,
}
```

3.1.1.7. 手动消息确认

只有在点对点模式且未开启事务的情况下，手动消息确认才会生效。通过设置**Session.CLIENT_ACKNOWLEDGE**参数，设置消息确认机制为客户端手动确认。并在消费消息时，通过**javax.jms.Message**的**acknowledge()**方法进行手动消息消费确认。设置参考如下：

```
Session session = connection.createSession(false,  
Session.CLIENT_ACKNOWLEDGE);  
  
...  
  
TextMessage textMessage=((TextMessage)message);  
  
textMessage.acknowledge();
```

说明：`createSession()`第一个参数为是否开启事务，默认为**false**；第二个参数为消息确认模式，默认为**AUTO ACKNOWLEDGE**（自动提交）。

验证:

通过**tong-admin topics stats <topic_name>**命令查看设置是否生效，当消息消费成功，并提交ACK，subscriptions中的**msgBacklog**可以看到消息积压情况。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t

```
"subscriptions" : {
  "jms-queue" : {
    "msgRateOut" : 0.0,
    "msgThroughputOut" : 0.0,
    "bytesOutCounter" : 744,
    "msgOutCounter" : 4,
    "msgRateRedeliver" : 0.0,
    "messageAckRate" : 0.0,
    "chunkedMessageRate" : 0,
    "msgBacklog" : 0,
    "backlogSize" : 0,
    "earliestMsgPublishTimeInBacklog" : 0,
    "msgBacklogNoDelayed" : 0,
    "blockedSubscriptionOnUnackedMsgs" : false,
    "msgDelayed" : 0,
    "unackedMessages" : 0,
    "type" : "Shared",
    "msgRateExpired" : 0.0,
    "totalMsgExpired" : 0,
    "lastExpireTimestamp" : 0,
    "lastConsumedFlowTimestamp" : 1726797302172,
    "lastConsumedTimestamp" : 1726797302175,
    "lastAckedTimestamp" : 1726797302344,
    "lastMarkDeleteAdvancedTimestamp" : 1726797302345,
    "consumers" : [ {
      "msgRateOut" : 0.0,
      "msgThroughputOut" : 0.0,
      "bytesOutCounter" : 186,
      "msgOutCounter" : 1,
      "msgRateRedeliver" : 0.0,
      "messageAckRate" : 0.0,
      "chunkedMessageRate" : 0.0,
      "consumerName" : "c7aa5",
      "availablePermits" : 999,
      "unackedMessages" : 0,
```

3.1.2. 发布订阅模式

3.1.2.1. 引入maven依赖

```
<dependency>

  <groupId>com.datastax.oss</groupId>

  <artifactId>pulsar-jms-all</artifactId>

  <version></version>

</dependency>
```

3.1.2.2. 客户端factory

```
Map<String, Object> properties = new HashMap<>();
```

```
properties.put("brokerServiceUrl", "pulsar://服务器IP:6650");

properties.put("webServiceUrl", http://服务器IP:8080);

PulsarConnectionFactory factory = new PulsarConnectionFactory(properties);
```

说明:

- brokerServiceUrl: 用于设置TongLINK/Q-CN Broker的端口和地址，如果是集群部署，可用逗号隔开；
- webServiceUrl: 用于设置TongLINK/Q-CN服务对外提供的http连接地址，如果是集群部署，可用逗号隔开。

3.1.2.3. 发送消息

```
Connection connection = factory.createConnection();

try {

    Session session = connection.createSession();

    Destination destination = session.createQueue(topic_name);

    MessageProducer messageProducer = session.createTopic(destination);

    TextMessage textMessage = session.createTextMessage("");

    messageProducer.send(textMessage);

} finally {

    connection.close();

}
```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.1.2.4. 消息发送验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否发送成功，**msgInCounter**为当前发送的消息数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t


```

    }

}

});

} catch (JMSEException e) {

    // TODO Auto-generated catch block

}

```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.1.2.6. 订阅消息验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否成功消费，**msgOutCounter**为当前消息消费的数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t

```

{
  "msgRateIn" : 0.0,
  "msgThroughputIn" : 0.0,
  "msgRateOut" : 0.0,
  "msgThroughputOut" : 0.0,
  "bytesInCounter" : 744,
  "msgInCounter" : 4,
  "bytesOutCounter" : 2232,
  "msgOutCounter" : 12,
  "averageMsgSize" : 0.0,
  "msgChunkPublished" : false,
  "storageSize" : 744,
  "backlogSize" : 186,
  "publishRateLimitedTimes" : 0,
  "earliestMsgPublishTimeInBacklogs" : 0,
  "offloadedStorageSize" : 0,
  "lastOffloadLedgerId" : 0,
  "lastOffloadSuccessTimeStamp" : 0,
  "lastOffloadFailureTimeStamp" : 0,
  "ongoingTxnCount" : 0,
  "abortedTxnCount" : 0,
  "committedTxnCount" : 0,

```

3.1.3. 事务

JMS1.1支持客户端开启事务。客户端开启事务的参数有以下两处:

1. **factory**的**jms.emulateTransactions**参数，需设置为**true**；
2. **createSession()**的第一个参数，需设置为**true**。

3.1.3.1. 生产者开启事务

生产者开启事务之后，需要对session进行手动提交，通过**session.rollback()**方法进行回滚。

```
Map<String, Object> properties = new HashMap<>();

properties.put("brokerServiceUrl", "pulsar://服务器IP:6650");

properties.put("webServiceUrl", "http://服务器IP:8080");

properties.put("jms.emulateTransactions", "true");

PulsarConnectionFactory factory = new PulsarConnectionFactory(properties);

Connection connection = factory.createConnection();

Session session = connection.createSession(true, Session.AUTO_ACKNOWLEDGE);

try {

    Destination destination = session.createQueue(topic_name);

    MessageProducer messageProducer = session.createProducer(destination);

    TextMessage textMessage = session.createTextMessage("");

    messageProducer.send(textMessage);

    session.commit();

} catch (Exception e) {

    e.printStackTrace();

    session.rollback();

} finally {

    connection.close();

}
```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.1.3.2. 消费者开启事务

事务的优先级高于消息确认，事务提交，消息确认也会提交，如果事务回滚或者未提交，哪怕手动进行了消息确认，也不会真正消费消息。

```
Map<String, Object> properties = new HashMap<>();

properties.put("brokerServiceUrl", "pulsar://服务IP:6650");

properties.put("webServiceUrl", "http://服务IP:8080");

properties.put("jms.emulateTransactions", "true");

PulsarConnectionFactory factory = new PulsarConnectionFactory(properties);

Connection connection = factory.createConnection();

connection.start();

Session session = connection.createSession(true, Session.AUTO_ACKNOWLEDGE);

try {

    Destination destination = session.createQueue(topic_name);

    MessageConsumer messageConsumer = session.createConsumer(destination);

    TextMessage message = (TextMessage) messageConsumer.receive();

    //消息处理

    session.commit();

}catch (Exception e){

    session.rollback();

}

connection.close();
```

说明：

topic_name实例：persistent://test-t/test-n/test-t

3.2. JMS2.0

3.2.1. 点对点

3.2.1.1. 引入maven依赖

```
<dependency>

    <groupId>com.datastax.oss</groupId>

    <artifactId>pulsar-jms-all</artifactId>

    <version></version>

</dependency>
```

3.2.1.2. 客户端factory

```
Map<String, Object> properties = new HashMap<>();

properties.put("brokerServiceUrl", "pulsar://服务器IP:6650");

properties.put("webServiceUrl", http://服务器IP:8080);

PulsarConnectionFactory factory = new PulsarConnectionFactory(properties);
```

说明：

- brokerServiceUrl：用于设置TongLINK/Q-CN Broker的端口和地址，如果是集群部署，可用逗号隔开；
- webServiceUrl：用于设置TongLINK/Q-CN服务对外提供的http连接地址，如果是集群部署，可用逗号隔开。

3.2.1.3. 发送消息

```
try (JMSContext context = factory.createContext()){

    Queue queue = context.createQueue(topic_name);

    context.createProducer().send(queue, "hello, test jms2.0");

}
```

```
} catch (JMSRuntimeException e) {  
  
    e.printStackTrace();  
  
}
```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.2.1.4. 消息发送验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否发送成功，**msgInCounter**为当前发送的消息数量。

示例如下：

tong-admin topics stats persistent://test-t/test-n/test-t

```
{
  "msgRateIn" : 0.0,
  "msgThroughputIn" : 0.0,
  "msgRateOut" : 0.0,
  "msgThroughputOut" : 0.0,
  "bytesInCounter" : 744,
  "msgInCounter" : 4,
  "bytesOutCounter" : 2232,
  "msgOutCounter" : 12,
  "averageMsgSize" : 0.0,
  "msgChunkPublished" : false,
  "storageSize" : 744,
  "backlogSize" : 186,
  "publishRateLimitedTimes" : 0,
  "earliestMsgPublishTimeInBacklogs" : 0,
  "offloadedStorageSize" : 0,
  "lastOffloadLedgerId" : 0,
  "lastOffloadSuccessTimeStamp" : 0,
  "lastOffloadFailureTimeStamp" : 0,
  "ongoingTxnCount" : 0,
  "abortedTxnCount" : 0,
  "committedTxnCount" : 0,
}
```

3.2.1.5. 订阅消息

```
try (JMSContext context = factory.createContext()){

    Queue queue = context.createQueue(topic_name);

    JMSConsumer consumer = context.createConsumer(queue);

    consumer.setMessageListener(new MessageListener() {
```

```
@Override

public void onMessage(Message message) {

    try {

        //消息处理

    } catch (JMSException e) {

        throw new RuntimeException(e);

    }

}

});
```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.2.1.6. 订阅消息验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否成功消费，**msgOutCounter**为当前消息消费的数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t

```
{
  "msgRateIn" : 0.0,
  "msgThroughputIn" : 0.0,
  "msgRateOut" : 0.0,
  "msgThroughputOut" : 0.0,
  "bytesInCounter" : 744,
  "msgInCounter" : 4,
  "bytesOutCounter" : 2232,
  "msgOutCounter" : 12,
  "averageMsgSize" : 0.0,
  "msgChunkPublished" : false,
  "storageSize" : 744,
  "backlogSize" : 186,
  "publishRateLimitedTimes" : 0,
  "earliestMsgPublishTimeInBacklogs" : 0,
  "offloadedStorageSize" : 0,
  "lastOffloadLedgerId" : 0,
  "lastOffloadSuccessTimeStamp" : 0,
  "lastOffloadFailureTimeStamp" : 0,
  "ongoingTxnCount" : 0,
  "abortedTxnCount" : 0,
  "committedTxnCount" : 0,
```

3.2.1.7. 手动消息确认

只有点对点模式才能进行手动消息确认。通过设置**Session.CLIENT_ACKNOWLEDGE**参数，设置消息确认机制为客户端手动确认。并在消费消息时，通过javax.jms.Message的acknowledge()方法进行手动消息消费确认。设置参考如下：

```
JMSContext context = factory.createContext(Session.CLIENT_ACKNOWLEDGE);

...

TextMessage textMessage=((TextMessage)message);

textMessage.acknowledge();
```

说明： createSession()第一个参数是为是否开启事务，默认为**false**；第二个参数为消息确认模式，默认为**AUTO_ACKNOWLEDGE**（自动提交）。

验证：

通过**tong-admin topics stats <topic_name>**命令查看设置是否生效，当消息消费成功，并提交ACK，subscriptions中的msgBacklog可以看到消息积压情况。

示例如下：

```
tong-admin topics stats persistent://test-t/test-n/test-t
```

```
"subscriptions" : {
  "jms-queue" : {
    "msgRateOut" : 0.0,
    "msgThroughputOut" : 0.0,
    "bytesOutCounter" : 744,
    "msgOutCounter" : 4,
    "msgRateRedeliver" : 0.0,
    "messageAckRate" : 0.0,
    "chunkedMessageRate" : 0,
    "msgBacklog" : 0,
    "backlogSize" : 0,
    "earliestMsgPublishTimeInBacklog" : 0,
    "msgBacklogNoDelayed" : 0,
    "blockedSubscriptionOnUnackedMsgs" : false,
    "msgDelayed" : 0,
    "unackedMessages" : 0,
    "type" : "Shared",
    "msgRateExpired" : 0.0,
    "totalMsgExpired" : 0,
    "lastExpireTimestamp" : 0,
    "lastConsumedFlowTimestamp" : 1726797302172,
    "lastConsumedTimestamp" : 1726797302175,
    "lastAckedTimestamp" : 1726797302344,
    "lastMarkDeleteAdvancedTimestamp" : 1726797302345,
    "consumers" : [ {
      "msgRateOut" : 0.0,
      "msgThroughputOut" : 0.0,
      "bytesOutCounter" : 186,
      "msgOutCounter" : 1,
      "msgRateRedeliver" : 0.0,
      "messageAckRate" : 0.0,
      "chunkedMessageRate" : 0.0,
      "consumerName" : "c7aa5",
      "availablePermits" : 999,
      "unackedMessages" : 0,
```

3.2.2. 发布订阅模式

3.2.2.1. 引入maven依赖

```
<dependency>

  <groupId>com.datastax.oss</groupId>

  <artifactId>pulsar-jms-all</artifactId>

  <version></version>

</dependency>
```

3.2.2.2. 客户端factory

```
Map<String, Object> properties = new HashMap<>();
```

```
properties.put("brokerServiceUrl", "pulsar://服务器IP:6650");

properties.put("webServiceUrl", http://服务器IP:8080);

PulsarConnectionFactory factory = new PulsarConnectionFactory(properties);
```

说明:

- brokerServiceUrl: 用于设置TongLINK/Q-CN Broker的端口和地址，如果是集群部署，可用逗号隔开；
- webServiceUrl: 用于设置TongLINK/Q-CN服务对外提供的http连接地址，如果是集群部署，可用逗号隔开。

3.2.2.3. 发送消息

```
try (JMSContext context = factory.createContext()){

    Topic topic = context.createTopic(topic_name);

    context.createProducer().send(topic, "hello, test jms2.0");

} catch (JMSRuntimeException e) {

    e.printStackTrace();

}
```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.2.2.4. 消息发送验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否发送成功，**msgInCounter**为当前发送的消息数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t


```
Thread.sleep(20000);

} catch (JMSRuntimeException e) {

    e.printStackTrace();

} catch (InterruptedException e) {

    throw new RuntimeException(e);

}
```

说明:

topic_name实例: persistent://test-t/test-n/test-t

3.2.2.6. 订阅消息验证

使用**tong-admin topics stats <topic_name>**命令查看消息是否成功消费, **msgOutCounter**为当前消息消费的数量。

示例如下:

tong-admin topics stats persistent://test-t/test-n/test-t

```
{
  "msgRateIn" : 0.0,
  "msgThroughputIn" : 0.0,
  "msgRateOut" : 0.0,
  "msgThroughputOut" : 0.0,
  "bytesInCounter" : 744,
  "msgInCounter" : 4,
  "bytesOutCounter" : 2232,
  "msgOutCounter" : 12,
  "averageMsgSize" : 0.0,
  "msgChunkPublished" : false,
  "storageSize" : 744,
  "backlogSize" : 186,
  "publishRateLimitedTimes" : 0,
  "earliestMsgPublishTimeInBacklogs" : 0,
  "offloadedStorageSize" : 0,
  "lastOffloadLedgerId" : 0,
  "lastOffloadSuccessTimeStamp" : 0,
  "lastOffloadFailureTimeStamp" : 0,
  "ongoingTxnCount" : 0,
  "abortedTxnCount" : 0,
  "committedTxnCount" : 0,
```

附录 A: 术语说明

名称	说明
TongLINK/Q-CN (简称TLQ-CN)	东方通云原生消息中间件
Broker	服务端工作节点
Client	客户端
Topic	主题
Message	消息
Producer	生产者，即消息的提供者
Consumer	消费者，即消息的消费者
Pub-Sub	发布及订阅
Point-to-Point(PTP)	点对点模式，即生产者和消费者之间的消息往来

